

Sistema de Chamados de TI

Fila de Prioridade — Implementação com Lista Encadeada Ordenada

Disciplina: Estrutura de Dados | Linguagem: Java

1. Introdução

Este relatório documenta o desenvolvimento de um sistema de gerenciamento de chamados para o departamento de Tecnologia da Informação de uma empresa. O sistema utiliza uma **fila de prioridade** implementada manualmente por meio de **lista encadeada ordenada**, sem o uso de estruturas prontas como `java.util.PriorityQueue`.

O objetivo é simular o atendimento de chamados técnicos respeitando dois critérios: nível de urgência definido pelo usuário e, como critério de desempate, a ordem de chegada do chamado ao sistema.

2. Estrutura do Projeto

O projeto é organizado em três classes dentro do pacote **chamados**:

- **Chamado.java** — entidade que representa um chamado (prioridade, descrição e timestamp).
- **SistemaDeChamado.java** — implementação da fila de prioridade com lista encadeada ordenada.
- **Main.java** — interface de console com menu interativo para o operador do sistema.

Estrutura de diretórios compatível com IntelliJ IDEA (clone + Run direto):

```
chamados/  
  chamados.iml  
  .idea/  
    modules.xml  
  src/  
    chamados/  
      Chamado.java  
      SistemaDeChamado.java  
      Main.java
```

3. Estrutura de Dados — Lista Encadeada Ordenada

A fila de prioridade é mantida como uma **lista encadeada simples sempre ordenada**. Cada nó armazena um objeto *Chamado* e uma referência ao próximo nó. A lista é mantida em ordem crescente de prioridade; em caso de empate, em ordem crescente de timestamp (o mais antigo vem primeiro).

Essa abordagem garante que a operação de remoção do próximo chamado seja feita em tempo constante $O(1)$, pois o elemento de maior prioridade está sempre na cabeça da lista.

Operação	Complexidade	Descrição
enfileirar()	$O(n)$	Percorre a lista para inserir na posição certa
proximo()	$O(1)$	Remove e retorna a cabeça da lista
consultarProximo()	$O(1)$	Lê a cabeça sem remover
tamanho()	$O(1)$	Retorna o contador interno
exibirChamadosPendentes()	$O(n)$	Percorre todos os nós

4. Classes Implementadas

4.1 Chamado.java

Representa a entidade central do sistema. Cada chamado possui:

- **prioridade** — inteiro onde menor valor indica maior urgência.
- **descricao** — texto livre descrevendo o problema.
- **timestamp** — valor em nanosegundos capturado no momento da criação, usado exclusivamente para desempate entre chamados de mesma prioridade.

4.2 SistemaDeChamado.java

Implementa a fila de prioridade por meio de uma lista encadeada com nó interno (*No*). O método **enfileirar()** percorre a lista e insere o novo chamado na posição correta, mantendo a ordenação. Os métodos **proximo()** e **consultarProximo()** operam diretamente na cabeça da lista, garantindo $O(1)$.

Trecho do método de inserção ordenada:

```
private boolean deveVirAntes(Chamado a, Chamado b) {
    if (a.getPrioridade() != b.getPrioridade()) {
        return a.getPrioridade() < b.getPrioridade();
    }
    return a.getTimestamp() < b.getTimestamp();
}
```

4.3 Main.java

Interface de linha de comando que oferece um menu com cinco operações ao operador:

- Registrar novo chamado (informa prioridade e descrição).
- Atender próximo chamado (remove da fila e exhibe).
- Consultar próximo sem atender (apenas visualiza).
- Listar todos os chamados pendentes na ordem de atendimento.
- Ver quantidade de chamados aguardando.

5. Como Executar

O projeto é compatível com IntelliJ IDEA Community ou Ultimate, sem dependências externas. Basta seguir os passos:

- Clonar o repositório ou descompactar o projeto.
- Abrir a pasta raiz no IntelliJ IDEA (*File* → *Open*).
- Aguardar a indexação; o módulo **chamados.iml** é reconhecido automaticamente.
- Definir o JDK (11 ou superior) se solicitado.
- Localizar a classe **Main** e pressionar o botão *Run* (Shift+F10).

6. Exemplo de Uso

Sequência de operações demonstrando o comportamento da fila:

```
Adicionar: prioridade=3, "Impressora não responde"
Adicionar: prioridade=1, "Servidor fora do ar"
Adicionar: prioridade=2, "VPN com lentidão"
Adicionar: prioridade=1, "Email corporativo caiu"    ← mesmo nível que servidor

Listar pendentes:
  1. [Prioridade 1] Servidor fora do ar          ← chegou primeiro
  2. [Prioridade 1] Email corporativo caiu      ← desempate por timestamp
  3. [Prioridade 2] VPN com lentidão
  4. [Prioridade 3] Impressora não responde

Atender: [Prioridade 1] Servidor fora do ar
Atender: [Prioridade 1] Email corporativo caiu
```

O exemplo confirma que a ordem de chegada prevalece quando dois chamados possuem o mesmo nível de prioridade.

7. Conclusão

O sistema atende integralmente aos requisitos propostos: fila de prioridade implementada sem uso de *PriorityQueue*, desempate por ordem de chegada, e interface de console funcional. A escolha da lista encadeada ordenada simplifica a lógica de remoção ($O(1)$) ao custo de inserção linear ($O(n)$), adequado para volumes típicos de chamados de suporte de TI.

A estrutura do projeto foi configurada para execução imediata no IntelliJ IDEA, sem necessidade de configuração adicional.